Assignment 3: Neural Networks

Course: CS4101 | CS6103

Deadline: 27th Oct, 2026

Total Marks: 20

Instructions

1. Work in groups of three, with the same members as Assignment 1.
2. Write properly indented, commented, and well-structured code.
3. Plagiarism results in zero for all involved group members.
4. Submit one ZIP file named `rollno1_rollno2_rollno3.zip`.
5. Upload the assignment to: `https://forms.gle/cCaYeAobRhzy8TJK8`.

Problem Statement

Implement a feedforward neural network and backpropagation from scratch to classify CIFAR-10 images. Experiment with hidden-layer widths and activation functions, and evaluate your selected model.

Use fully connected layers only. Convolutional networks, pretrained models, and transfer learning are not permitted.

Dataset and Preprocessing

CIFAR-10 contains 50,000 training images and 10,000 test images. Each image is a 32×32 RGB image from one of ten classes.

Label	Class	Label	Class
0	Airplane	5	Dog
1	Automobile	6	Frog
2	Bird	7	Horse
3	Cat	8	Ship
4	Deer	9	Truck

Dataset reference: `https://keras.io/api/datasets/cifar10/`.

```
from keras.datasets import cifar10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
```

1. Create a random number generator using `np.random.default_rng(42)`. Process classes in label order (0–9). For each class, obtain its original training indices in ascending order and select 1,000 using `rng.choice(class_indices, size=1000, replace=False)`. Use the first 800 selected indices for training and the remaining 200 for validation. Concatenate subsets in class order without resetting the generator between classes. Reuse the resulting 8,000 training and 2,000 validation images for all experiments.
2. Convert pixel values to floating point and divide by 255 to scale them to $[0, 1]$. Do not apply further normalization or standardization.
3. Retain all colour channels and flatten each image into a vector of 3,072 features. Encode labels as needed.

4. Reserve all 10,000 official test images for final evaluation. Do not use them for hyperparameter tuning.

Question 1: Implement Forward Propagation

Build a configurable feedforward neural network using NumPy.

- Accept 3,072 input features and support one or two hidden layers.
- Allow the hidden-layer widths to be specified.
- Support sigmoid and ReLU hidden-layer activations.
- Use ten output neurons with a numerically stable softmax.
- Choose and document your parameter initialization method.

For a batch of B images, return class probabilities of shape $(B, 10)$. Verify that each row contains valid probabilities summing to one. State the weight and bias shapes and calculate the number of trainable parameters for $3072 \to 64 \to 10$.

Question 2: Implement Backpropagation and Training

Implement backpropagation manually using the mean categorical cross-entropy loss:

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^{B} \sum_{c=1}^{10} y_{ic} \log \hat{p}_{ic}.$$

Here, y_{ic} is the one-hot target and $\hat{p}_{ic}$ is the predicted probability. Compute the loss numerically stably.

(a) Compute gradients for every weight matrix and bias vector.

(b) Implement mini-batch SGD and momentum-based mini-batch gradient descent. For momentum, use

$$v_t = 0.9 v_{t-1} + \nabla_\theta \mathcal{L}_{\mathcal{B}_t}(\theta_{t-1}), \qquad \theta_t = \theta_{t-1} - \eta v_t.$$

Initialize each velocity to zero at the start of a run and retain it across batches and epochs. Here, $\mathcal{L}_{\mathcal{B}_t}$ is the mean mini-batch loss and η is the learning rate.

(c) Shuffle the training data before each epoch. Set and document random seeds for initialization and shuffling.

(d) After each epoch, evaluate the current model on the complete training and validation subsets. Record loss and accuracy for both subsets.

(e) Demonstrate a complete training run. Choose and document the architecture, optimizer, learning rate, batch size, and number of epochs.

Keep the implementation configurable so that experiments do not require rewriting the training loop.

Hyperparameter Tuning

1. Choose at least three distinct hidden-layer widths (numbers of hidden units), such as 32, 64, and 128.
2. Evaluate both sigmoid and ReLU at each chosen width, giving at least six configurations. Keep other hyperparameters fixed for these comparisons. If using two hidden layers, use the chosen width and activation in both layers.

Question 3: Present and Interpret the Results

(a) Track your experiments using Weights & Biases (`wandb`). Present a table containing each configuration, parameter count, best validation accuracy, and training time. Briefly state how training time was measured.

(b) Select the model with the highest validation accuracy. Within a run, retain the earliest checkpoint achieving that accuracy. For ties between runs, prefer fewer parameters, followed by the earlier run.

(c) Plot training and validation loss and accuracy against epoch for the selected run. Include a plot comparing the best validation accuracies across runs.

(d) Compare the effects of activation function and hidden-layer width. Discuss whether the learning curves indicate underfitting or overfitting.

Question 4: Evaluate the Selected Model

Restore the selected checkpoint and evaluate it on the official test set.

(a) Report test accuracy.

(b) Plot a 10×10 confusion matrix with class names and clearly labelled true and predicted axes.

(c) Briefly explain the limitations of using flattened pixels with a fully connected network for CIFAR-10 classification.

Do not change the model or hyperparameters based on test results.

Implementation and Submission

- Use Python and NumPy for the network, loss, gradients, and optimizer updates. Keras may be used only to load CIFAR-10. Do not use automatic differentiation or pre-built model-training components.
- You may use Pandas, Matplotlib, Seaborn, and scikit-learn for tables, plots, and evaluation metrics.
- Submit exactly one implementation notebook (`.ipynb`) and one PDF report inside the ZIP file. Include group members' names and roll numbers in both files.
- Display all required results in the notebook and include the experiment results, figures, explanations, and `wandb` project link in the report.
- The notebook must run from beginning to end. Document dependencies, random seeds, and execution instructions. Numerical results should reproduce within reasonable floating-point tolerance; execution times may vary.